GYANBIT

RETRO ARCADE ADVENTURE

OFFICIAL HANDBOOK

DESIGNED IN INDIA • ENGINEERED FOR THE WORLD

UNAUTHORIZED REPRODUCTION PROHIBITED

REV 3.1.1

MMCOE GYANBIT LABS

EVERYTHING YOU NEED TO BUILD & PLAY

01. PHILOSOPHY

GyanBit was born from a singular mission: to strip away the "black box" of modern consumer electronics. In a world of proprietary software and locked bootloaders, GyanBit is a radical act of openness. This console does not belong to the manufacturer; it belongs to the programmer who builds with it.

THE CREATOR'S JOURNEY

Unlike standard consoles where you are a consumer of media, GyanBit asks you to be the architect. We combine the power of modern web technologies with the gritty reality of bare-metal embedded circuits. Whether you are learning your first loop in JavaScript or optimizing frequency cycles in MicroPython, GyanBit scales with your curiosity.

MISSION OBJECTIVE

Provide every student with the tools to move from consumption to creation, utilizing sustainable materials and open protocols.

02. HARDWARE

The heart of GyanBit is the Raspberry Pi RP2040. This dual-core microcontroller provides the horsepower needed to drive a monochrome OLED at a smooth 30 frames per second while handling input interrupts and PWM audio.

03. ARCHITECTURE

GyanBit Studio runs a custom JavaScript-to-MicroPython transpiler. This allows you to code in the familiar language of the web (JS) while producing fast code for the console.

THE RUNTIME LIFECYCLE

01 **Validation:** Checks for errors in your code.

02 **Transpilation:** Your JS code becomes Python code.

03 **Upload:** The code travels through the USB cable to the console.

04 **Execution:** The console starts playing your game instantly!

THE 30 FPS LOOP

GyanBit runs your code 30 times every single second! This is what makes movements look smooth. In every "tick," we check buttons, move players, and draw the screen.

```
bit.loop(() => {
    // 1. Check buttons (Input)
    // 2. Clear previous frame
    // 3. Draw new pixels
});
```

04. API GUIDE

The `bit` object is like a magic box of tools. You use it to tell the screen what to show and hear what the buttons are doing.

DRAWING TOOLS

bit.clear()	Clean the screen. Like washing a chalkboard!
bit.dot(x, y)	Draw one tiny point.
bit.line(x1,y1,x2,y2)	Connect two points with a line.
bit.fill(x,y,w,h)	Draw a solid color box.
bit.fillCircle(c,r)	Draw a solid circle (for balls or players).

BUTTON TOOLS

```
if (bit.isHeld('up')) {
  // Do something when Up is pressed!
}
```

SOUND TOOLS

bit.beep(hz, ms)	Makes a "BEEP" sound. High HZ = Squeaky, Low HZ = Grumbly.

05. BOUNCING BALL

Let's make a ball that bounces around the screen! We need to remember three things: Where it is, and how fast it is moving.

1. SET THE START

```
let x = 64;  // Left-Right position
let y = 32;  // Up-Down position
let dx = 2;  // Speed (Left-Right)
let dy = 2;  // Speed (Up-Down)
```

2. MAKE IT MOVE

```
bit.loop(() => {
    bit.clear();

    // Step 1: Update position
    x += dx;
    y += dy;

    // Step 2: Hit a wall? Turn around!
    if (x <= 0 || x >= 127) dx = -dx;
    if (y <= 0 || y >= 63)  dy = -dy;

    // Step 3: Draw the ball!
    bit.fillCircle(x, y, 4);
});
```

06. PADDLE POWER

Now, let's make the game interactive! We are going to add a **Paddle** that YOU can move using the buttons. It's like having a digital tennis racket!

1. CREATE THE PADDLE

We need to tell the GyanBit where the paddle starts. Think of `paddleY` as the "Up and Down" position of your racket.

```
let paddleY = 24;       // Start near the middle
const PADDLE_HEIGHT = 16;
```

2. LISTENING TO BUTTONS

We want the paddle to move when we press the **UP** and **DOWN** buttons. We also need to make sure it doesn't float off the screen!

```
// Move UP if the button is held
if (bit.isHeld('up') && paddleY > 0) {
    paddleY -= 3;
}

// Move DOWN if the button is held
if (bit.isHeld('down') && paddleY < 64 - PADDLE_HEIGHT) {
    paddleY += 3;
}

// Draw the paddle at the left edge
bit.fill(0, paddleY, 4, PADDLE_HEIGHT);
```

07. THE BOUNCE TEST

This is the most important part! We want to tell the ball: "If you touch my paddle, bounce back!"

THE LOGIC

We check if the ball is hitting the left side AND if it is at the same height as our paddle. If both are true... **PING!**

```
if (x ≤ 4 && y ≥ paddleY && y ≤ paddleY + PADDLE_HEIGHT) {
    dx = -dx;          // Hit the paddle! Turn around!
    bit.beep(800, 50); // Play a fun "Ping" sound
}
```

HOW COLLISION WORKS

"When the Ball hits the Paddle, it triggers a Bounce!"

08. GAME GALLERY

Want to see what's possible? Check out the built-in games! You can read their code to learn expert secrets.

SNAKE

Eat the dots and grow longer. Don't hit yourself!

SPACE INVADERS

Protect the planet from falling enemies!

FLAPPY BIRD

Flap your wings and fly through the pipes.

DOOM CLONE

Explore a 3D world on your tiny console!

PRO TIP

"Don't just play the games. Change them! Make the snake move super fast or make the bird jump extra high."

09. BYTE BOT AI

Stuck on a bug? Byte Bot is your personal coding teacher. Click the robot icon to start a chat!

HOW TO ASK FOR HELP

Byte Bot loves specific questions. Instead of "Fix my code," try these:

- "How do I make my ball move faster?"
- "Why is my paddle falling off the screen?"
- "Can you help me draw a star shape?"

BYTE BOT MESSAGE

"Always remember to clear your screen with `bit.clear()` before drawing something new!"

10. CARE & SAFETY

Your GyanBit is a tough little machine, but it likes to be treated with respect!

BAMBOO CARE

Your console is made of real bamboo! Keep it dry and clean with a soft cloth. Don't leave it in the hot sun.

BATTERY TIPS

Charge it using a standard USB cable. When the light goes out, it's ready for hours of fun!

YOU DID IT!

You are now a GyanBit Engineer. Go build something amazing!